Ulster University

JavaScript

Arrays and Conditional Statements

ulster.ac.uk

Arrays

What is an array?

- Arrays are generally described as "list-like objects"; they are basically single objects that contain multiple values stored in a list.

- Array objects can be stored in variables and dealt with in much the same way as any other type of value, the difference being that we can access each value inside the list individually.

- Maybe we've got a series of product items and their prices stored in an array, and we want to loop through them all and print them out on an invoice, while totalling all the prices together and printing out the total price at the bottom.

Arrays

- If we didn't have arrays, we'd have to store every item in a separate variable, then call the code that does the printing and adding separately for each item.
- This would be much longer to write out, less efficient, and more error-prone. If we had 10 items to add to the invoice it would already be annoying, but what about 100 items, or 1000?

Arrays

Syntaxes

- There are two syntaxes for creating an empty array:

```
let arr = new Array();
let arr = [];
```

- Almost all the time, the second syntax is used. We can supply initial elements in the brackets.

- Arrays can be declared using all the normal variable declarations:
  - var
  - let
  - const

Arrays

- Neither the length of a JavaScript array nor the types of its elements are fixed. Since an array's length can change at any time, and data can be stored at non-contiguous locations in the array.

- Arrays consist of square brackets and elements that are separated by commas.

```
let fruits = ["Apple", "Orange", "Plum"];
```

- Array elements are numbered, starting with zero.

Arrays

- We can get an element by its number in square brackets:

```
let fruits = ["Apple", "Orange", "Plum"];

alert( fruits[0] ); // Apple
alert( fruits[1] ); // Orange
alert( fruits[2] ); // Plum
```

- This is known as accessing an element at its index position.

Arrays

- We can also replace an element by its index position:

```
fruits[2] = 'Pear'; // now ["Apple", "Orange", "Pear"]
```

- Or add a new item to the array:

```
fruits[3] = 'Lemon'; // now ["Apple", "Orange", "Pear", "Lemon"]
```

Arrays

- The total count of the elements in the array is its length:

```
let fruits = ["Apple", "Orange", "Plum"];

alert( fruits.length ); // 3
```

- We can also use alert to show the whole array.

```
let fruits = ["Apple", "Orange", "Plum"];

alert( fruits ); // Apple,Orange,Plum
```

Arrays

Trailing Comma

- An array, just like an object, may end with a comma:

```
let fruits = [
  "Apple",
  "Orange",
  "Plum",
];
```

- The “trailing comma” style makes it easier to insert/remove items, because all lines become alike.

Arrays

Queue

- A queue is one of the most common uses of an array. In computer science, this means an ordered collection of elements which supports two operations:
  - Push appends an element to the end.
  - Shift get an element from the beginning, advancing the queue, so that the 2nd element becomes the 1st.

Arrays

Pop & Push – End of Array

- Pop extracts the last element of the array and returns it:

```
let fruits = ["Apple", "Orange", "Pear"];

alert( fruits.pop() ); // remove "Pear" and alert it

alert( fruits ); // Apple, Orange
```

- Push appends the element to the end of the array:

```
let fruits = ["Apple", "Orange"];

fruits.push("Pear");

alert( fruits ); // Apple, Orange, Pear
```

Arrays

Shift & Unshift – Start of Array

- Shift extracts the first element of the array and returns it:

```
let fruits = ["Apple", "Orange", "Pear"];

alert( fruits.shift() ); // remove Apple and alert it

alert( fruits ); // Orange, Pear
```

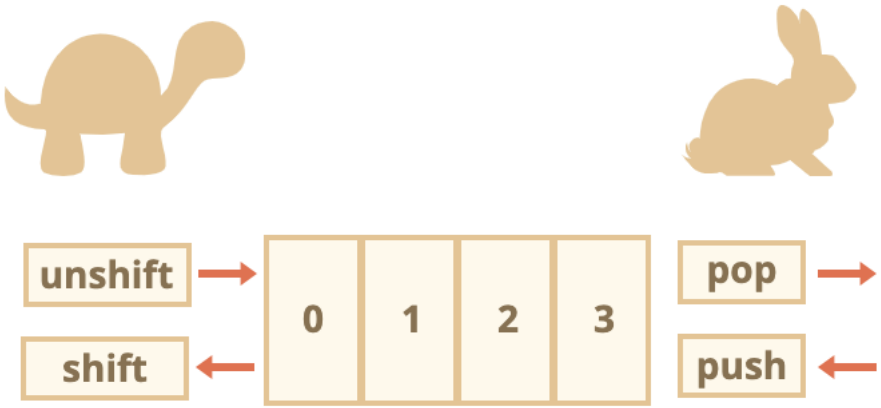
- Unshift adds the element to the beginning of the array:

```
let fruits = ["Orange", "Pear"];

fruits.unshift('Apple');

alert( fruits ); // Apple, Orange, Pear
```

Arrays

Performance

- Methods push/pop run fast, while shift/unshift are slow.



- Why is it faster to work with the end of an array than with its beginning?

Arrays

Performance

- Let's see what happens during the execution:

```
fruits.shift(); // take 1 element from the start
```

- It's not enough to take and remove the element with the number 0. Other elements need to be renumbered as well.

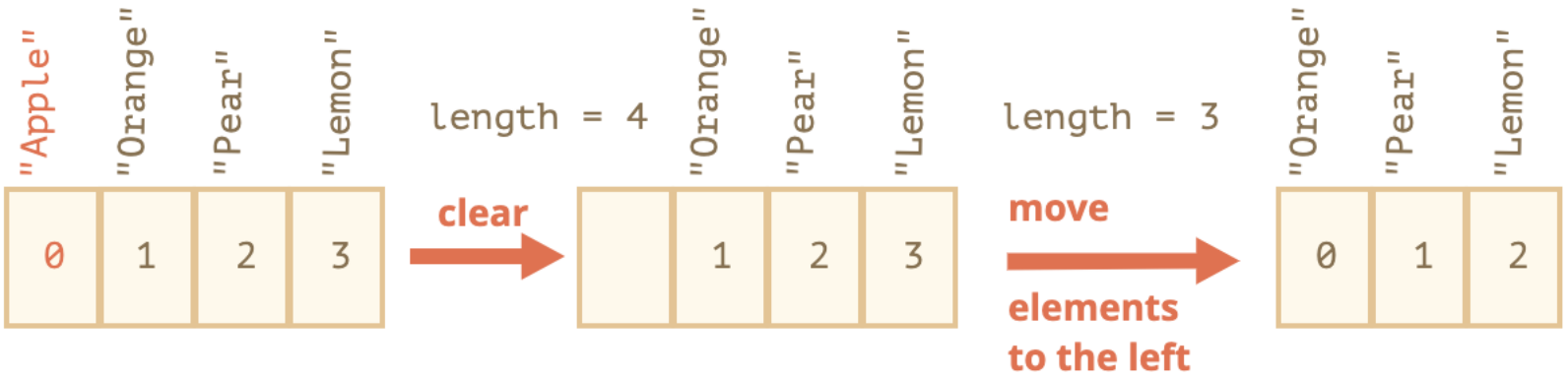
- The shift operation must do 3 things:
  1. Remove the element with the index 0.
  2. Move all elements to the left, renumber them from the index 1 to 0, from 2 to 1 and so on.
  3. Update the length property.

Arrays

Performance

- **The more elements in the array, the more time to move them, more in-memory operations.**

- The similar thing happens with unshift: to add an element to the beginning of the array, we need first to move existing elements to the right, increasing their indexes.

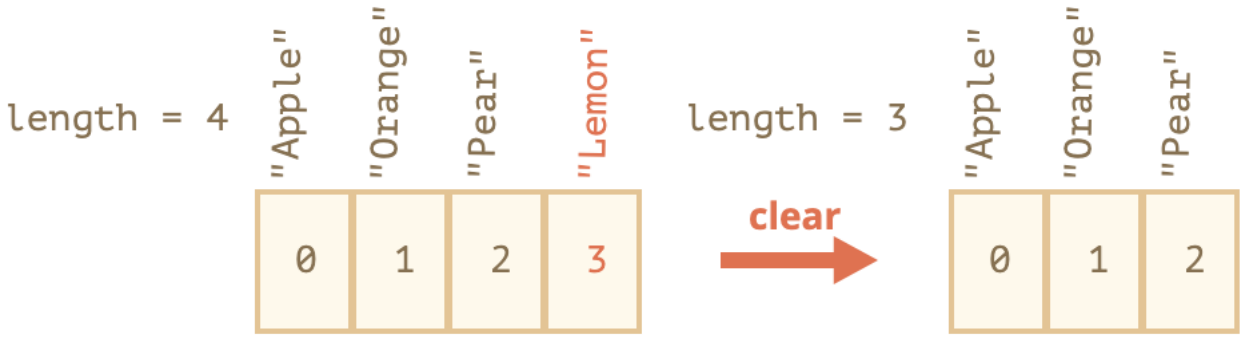


Arrays

Performance

- And what's with push/pop? They do not need to move anything.
- To extract an element from the end, the pop method cleans the index and shortens length.
- The actions for the pop operation:

```
fruits.pop(); // take 1 element from the end
```



Conditional Statements

Ulster University

If Statement

- Sometimes, we need to perform different actions based on different conditions. To do that, we can use the ”if” statement.

- The if(...) statement evaluates a condition in parentheses and, if the result is true, executes a block of code.

```
if (year == 2015) alert( 'You are right!' );
```

- In the example above, the condition is a simple equality check (year == 2015), but it can be much more complex.

If Statement

- If we want to execute more than one statement, we must wrap our code block inside curly braces:

```
if (year == 2015) {
  alert( "That's correct!" );
  alert( "You're so smart!" );
}
```

- We recommend wrapping your code block with curly braces { } every time you use an if statement, even if there is only one statement to execute.

- Doing so improves readability.

If Statement

Boolean Conversion

- The if (…) statement evaluates the expression in its parentheses and converts the result to a Boolean.
- A number 0, an empty string "", null, undefined, and NaN all become false. Because of that they are called “falsy” values.
- Other values become true, so they are called “truthy”.

If Statement

Boolean Conversion

- So, the code under this condition would never execute:

```
if (0) { // 0 is falsy
  ...
}
```

- …and inside this condition – it always will:

```
if (1) { // 1 is truthy
  ...
}
```

If Statement

Boolean Conversion

- We can also pass a pre-evaluated Boolean value to if, like this:

```
let cond = (year == 2015); // equality evaluates to true or false

if (cond) {
  ...
}
```

If Statement

Else Clause

- The if statement may contain an optional “else” block. It executes when the condition is false.

```
if (year == 2015) {
  alert( 'You guessed it right!' );
} else {
  alert( 'How can you be so wrong?' ); // any value except 2015
}
```

If Statement

Else-If

- Sometimes, we'd like to test several variants of a condition. The else if clause lets us do that.

```
if (year < 2015) {
  alert( 'Too early...' );
} else if (year > 2015) {
  alert( 'Too late' );
} else {
  alert( 'Exactly!' );
}
```

- In the code above, JavaScript first checks year < 2015. If that is falsy, it goes to the next condition year > 2015. If that is also falsy, it shows the last alert.

- There can be more else if blocks. The final else is optional.

Switch Statement

- A switch statement can replace multiple if checks.
- It gives a more descriptive way to compare a value with multiple variants.
- The switch has one or more case blocks and an optional default.

```
switch(x) {
  case 'value1':  // if (x === 'value1')
    ...
    [break]

  case 'value2':  // if (x === 'value2')
    ...
    [break]

  default:
    ...
    [break]
}
```

Switch Statement

- The value of x is checked for a strict equality to the value from the first case (that is, value1) then to the second (value2) and so on.
- If the equality is found, switch starts to execute the code starting from the corresponding case, until the nearest break (or until the end of switch).
- If no case is matched then the default code is executed (if it exists).

```
switch(x) {
  case 'value1':
    ...
    [break]

  case 'value2':
    ...
    [break]

  default:
    ...
    [break]
}
```

Switch Statement

An example of switch (the executed code is highlighted):

```
let a = 2 + 2;

switch (a) {
  case 3:
    alert( 'Too small' );
    break;
  case 4:
    alert( 'Exactly!' );
    break;
  case 5:
    alert( 'Too large' );
    break;
  default:
    alert( "I don't know such values" );
}
```

- Here the switch starts to compare a from the first case variant that is 3. The match fails.
- Then 4. That's a match, so the execution starts from case 4 until the nearest break.
- **If there is no break then the execution continues with the next case without any checks.**

Switch Statement

- An example without break:

```
let a = 2 + 2;

switch (a) {
  case 3:
    alert( 'Too small' );
  case 4:
    alert( 'Exactly!' );
  case 5:
    alert( 'Too big' );
  default:
    alert( "I don't know such values" );
}
```

Switch Statement

Type

- Let's emphasise that the equality check is always strict. The values must be of the same type to match.
- For 0, 1, the first alert runs.
- For 2 the second alert runs.
- But for 3, the result of the prompt is a string "3", which is not strictly equal === to the number 3. So we've got a dead code in case 3!
  The default variant will execute.

```
let arg = prompt("Enter a value?");
switch (arg) {
  case '0':
  case '1':
    alert( 'One or zero' );
    break;

  case '2':
    alert( 'Two' );
    break;

  case 3:
    alert( 'Never executes!' );
    break;
  default:
    alert( 'An unknown value' );
}
```

Questions?

Ulster University